



DOSSIER PROFESSIONNEL (DP)

Nom de naissance ➤ CARLI
Nom d'usage ➤ CARLI
Prénom ➤ Julie
Adresse ➤ 48 rue de la République 28110 Lucé

Titre professionnel visé

Concepteur Développeur d'Applications

MODALITÉ D'ACCÈS :

- ☒ Parcours de formation
- ☐ Validation des Acquis de l'Expérience (VAE)

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel. **Ce titre est délivré par le Ministère chargé de l'emploi.**

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente **obligatoirement à chaque session d'examen.**

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE.

Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel** (DP) dans lequel le candidat a consigné les preuves de sa pratique professionnelle
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels du ministère chargé de l'Emploi]

Ce dossier comporte :

- pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- une déclaration sur l'honneur à compléter et à signer ;
- des documents illustrant la pratique professionnelle du candidat (facultatif)
- des annexes, si nécessaire.

DOSSIER PROFESSIONNEL ^(DP)

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.



<http://travail-emploi.gouv.fr/titres-professionnels>

Sommaire

Exemples de pratique professionnelle

Développer une application sécurisée

p.

- Projet Miaouff

p. p. _____

p. _____

p. _____

Concevoir et développer une application sécurisée organisée en couches

p.

- Projet Miaouff

p. p. _____

p. _____

p. _____

Préparer le déploiement d'une application sécurisée

p.

- Projet Miaouff

p. p. _____

p. _____

p. _____

Titres, diplômes, CQP, attestations de formation *(facultatif)*

p. _____

Déclaration sur l'honneur

p. _____

Documents illustrant la pratique professionnelle *(facultatif)*

p. _____

Annexes *(Si le RC le prévoit)*

p. _____

EXEMPLES DE PRATIQUE PROFESSIONNELLE

Activité-type 1 Développer une application sécurisée

Exemple n°1 - Projet Miaouff

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Présentation du projet :

Le projet consiste à réaliser un site vitrine dédié aux refuges animaliers, nommé **Miaouff**. Ce site permet de mettre en avant les animaux à adopter, de promouvoir l'adoption responsable et d'informer le public sur les besoins spécifiques de chaque espèce ou race. En parallèle, une partie boutique en ligne permet aux utilisateurs d'acheter des produits pour animaux, dont les bénéfices peuvent contribuer à soutenir les refuges. Le site inclut également une section glossaire présentant différentes espèces et races (ex : husky, labrador, etc.), une section blog ainsi que des jeux éducatifs ou ludiques (un quizz avec des questions sur l'abandon, le bien-être animal, un memory...).

Les utilisateurs inscrits peuvent se connecter pour acheter des produits, tandis que les administrateurs disposent d'un accès sécurisé pour gérer l'ensemble des contenus du site. Les visiteurs non connectés peuvent consulter les informations librement.

Objectifs du projet :

Le but de ce projet est de favoriser l'adoption des animaux en sensibilisant le public aux réalités de l'abandon et aux besoins spécifiques des animaux. Miaouff souhaite également fournir une expérience complète autour de l'animal : du coup de cœur à l'adoption, en passant par l'apprentissage, le jeu et l'achat de produits utiles. Il s'agit de créer un pont entre information, action et soutien, tout en valorisant le rôle des refuges.

Limites du projet :

Cette version du projet représente une évolution significative de la plateforme, avec des améliorations apportées tant sur le plan fonctionnel que visuel. Malgré ces avancées, certains axes de développement restent envisageables afin d'enrichir davantage l'expérience utilisateur. Des évolutions futures pourraient notamment porter sur l'ajout de



nouvelles fonctionnalités, le renforcement des interactions entre utilisateurs ou encore l'optimisation continue des contenus et des services proposés.

Le projet Miaouff a été développé sur un ordinateur équipé de macOS, à l'aide de l'éditeur de code Visual Studio Code.

Le back-end repose sur le framework Flask (Python), avec l'intégration de plusieurs bibliothèques telles que Flask-Login pour la gestion des connexions utilisateurs, Flask-Mail pour l'envoi de courriels (notamment pour la réinitialisation de mot de passe), Flask-Session pour la gestion des sessions côté serveur, ainsi que Werkzeug pour le hachage des mots de passe.

Deux systèmes de base de données sont utilisés :

- PostgreSQL, associé à l'ORM SQLAlchemy, pour gérer les données principales du site (utilisateurs, produits, commandes, refuges, animaux, etc.).
- MongoDB Atlas, pour stocker et gérer les contenus de la section blog, plus adaptée à un modèle NoSQL.

Le front-end repose principalement sur du HTML et du CSS, avec une gestion serveur via Flask.

Le JavaScript est utilisé pour quelques fonctionnalités spécifiques comme la connexion des utilisateurs ou les jeux éducatifs. Le reste du site fonctionne principalement avec du HTML et du code Python côté serveur, ce qui simplifie la gestion et favorise la clarté du projet.

Le projet utilise Git pour le versionnement du code source, avec hébergement sur GitHub.

Le déploiement du site a été réalisé via Railway, avec une base de données PostgreSQL hébergée sur Neon.

Un fichier `.env` unique est utilisé pour stocker les variables sensibles (identifiants de connexion, clés secrètes, etc.) afin d'assurer la sécurité et la portabilité du projet.

Une automatisation simple a également été mise en place avec Make, afin que les utilisateurs qui remplissent le formulaire de contact Tally reçoivent un mail récapitulatif de leur demande, avec le délai de réponse. L'équipe de Miaouff reçoit aussi une notification de Tally via Discord. Un chatbot a été automatisé avec Gemini.



Cahier des charges

Objectifs du site :

Le site *Miaouff* a pour objectif de promouvoir l'adoption animale, de présenter les différentes espèces et races d'animaux, ainsi que de proposer une boutique en ligne associée. Il permet également aux administrateurs de gérer le contenu du site (animaux, refuges, produits, articles de blog, etc.) via des interfaces dédiées.

Cible du site :

Toute personne intéressée par l'adoption animale, la découverte d'informations sur les espèces, ou l'achat de produits pour animaux.

Type de site :

Site vitrine et e-commerce avec espace d'administration sécurisé.

Plan du site :

- **Page d'accueil** présentant le concept, les valeurs et les actions de *Miaouff*.
- **Section adoption** : liste des animaux à adopter, avec fiches détaillées.
- **Section espèces et races** : présentation d'informations sur chaque espèce et race d'animal.
- **Boutique en ligne** : produits pour animaux, avec panier et système de commande.
- **Section blog** : articles publiés par l'équipe, stockés dans une base NoSQL (MongoDB).
- **Section jeux** : mini-jeux éducatifs et ludiques autour du thème animalier.
- **Espace administrateur** : gestion des refuges, animaux, animaux de compagnie, produits, utilisateurs, catégories et articles de blog.
- **Formulaire de contact** intégré via Tally, avec automatisation Make.

- **Page de connexion** permettant l'accès à l'espace administrateur.

Aspect visuel du site :

Le design suit la charte graphique réalisée, en utilisant les couleurs principales #e66a4a pour du orange foncé et #6b8e23 pour du vert foncé, avec un style clair, accessible et cohérent sur l'ensemble du site. L'ergonomie a été pensée pour offrir une navigation intuitive.

1. Installer et configurer son environnement de travail en fonction du projet

Le projet *Miaouff* a été développé sur un ordinateur équipé de macOS, à l'aide de l'éditeur de code Visual Studio Code.

Le back-end repose sur le framework Flask (Python), avec deux systèmes de bases de données :

- PostgreSQL, associé à l'ORM SQLAlchemy, pour gérer les données principales du site (utilisateurs, produits, commandes, refuges, animaux, etc.).
- MongoDB Atlas, utilisé pour stocker et gérer les contenus de la section blog, plus adaptée à un modèle NoSQL.

Pour la partie front-end, des templates HTML ont été créés et stylisés à l'aide de CSS personnalisé, en suivant la charte graphique du projet, et complétés par du JavaScript pour certaines fonctionnalités interactives.

Le versionnement du code source est géré avec Git, avec un dépôt hébergé sur GitHub. Le déploiement du site est envisagé via Railway, associé à un nom de domaine acquis sur Gandi.net.

2. Développer des interfaces utilisateur

Pour créer une application web avec Flask, il est d'abord nécessaire de s'assurer que Flask est installé en utilisant l'installateur de paquets pip : ***pip install flask***.

Une fois Flask installé, on peut commencer à développer l'application en créant un fichier Python principal (dans mon cas, nommé **app.py**) :

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route('/')
def home():
    return render_template("home.html")

app.run(debug=True)
```

On utilise la fonction *render_template()* fournie par Flask, qui permet d'utiliser le moteur de modèles **Jinja** pour afficher les fichiers HTML présents dans le dossier **templates**. Cela permet de créer un fichier HTML pour chaque page de l'application.

Pour éviter de répéter du code commun à toutes les pages, comme le **CSS**, le **header** et le **footer**, on crée un fichier **base.html** que les autres fichiers HTML étendent avec `{% extends "base.html" %}`. Ainsi, chaque page hérite automatiquement de ces éléments présents sur tout le site, tout en pouvant ajouter son contenu spécifique.

Exemple de la page **contact.html** qui hérite de **base.html** :

```
templates > <> contact.html > ...
1  {% extends "base.html" %}
2
3  {% block content %}
4  <br>
5  <H1 class="contact-title">Nous contacter</H1>
6  <div class="contact-form">
7  <iframe data-tally-src="https://tally.so/embed/nPBXq1?alignLeft=1&hideTitle=1&transparentBackground=1&dynamicH
8  <script>var d=document,w="https://tally.so/widgets/embed.js",v=function(){if(typeof Tally?Tally.load
9  </div>
10
11  {% endblock %}
12
```

Extrait du code css de la page **contact.html** (dans le fichier **styles.css**)

```
553  /* Contact */
554
555  .contact-title {
556      color: #ff7e5f;
557      text-align: center;
558  }
559
560  .contact-form {
561      width: 750px;
562      margin: 0 auto;
563  }
564
```

Capture d'écran du site web de la page de contact équivalente :

The screenshot shows the 'Nous contacter' (Contact Us) page of the Miaouff website. The page has an orange header with the Miaouff logo and navigation links: Accueil, Refuges, Animaux, Produits, Conseils, Blog, Jeux. On the right of the header are links for 'Mon compte' and a shopping cart icon. The main content area is white and features the title 'Nous contacter' in orange. Below the title is a contact form with the following fields: 'Je m'appelle' (Name) with sub-fields for 'John' and 'Doe'; 'Mon e-mail' (Email) with the value 'johndoe@example.com'; 'Mon n° de téléphone' (Phone number) with the value '6 12 34 56 78'; 'Ma demande concerne' (Subject) with a dropdown menu; and 'Je détaille ma demande' (Details) with a large text area. Below the form is a checkbox for 'Afin de traiter ma demande, j'accepte que Miaouff stocke et traite mes données' (To process my request, I accept that Miaouff stores and processes my data). At the bottom of the form is an 'Envoyer' (Send) button with a right arrow. In the bottom right corner of the form area is a 'Made with Tally' badge. The footer is orange and contains links for 'Conditions générales de vente', 'Mentions légales', 'Politique de confidentialité', 'Politique de cookies', 'FAQ', and 'Contact'. It also includes social media icons for Instagram, Facebook, and TikTok, and the Miaouff logo.

Extrait du code de **base.html** :

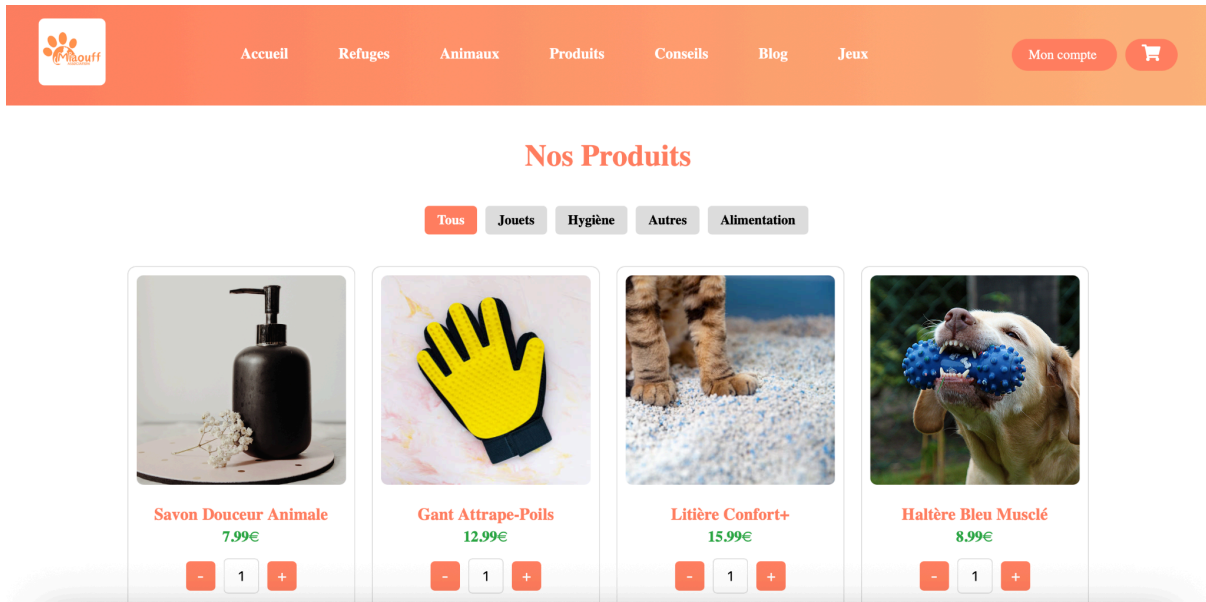
```
templates > <> base.html > ...
 2  <html lang="fr">
13  <body>
48  <div>
79  </div>
80  </div>
81
82  <!-- Footer -->
83  <footer class="footer">
84      <div class="footer-links">
85          <a href="{{ url_for('terms_conditions') }}">Conditions générales de vente</a>
86          <a href="{{ url_for('legal_notices') }}">Mentions légales</a>
87          <a href="{{ url_for('privacy_policy') }}">Politique de confidentialité</a>
88          <a href="{{ url_for('cookie_policy') }}">Politique de cookies</a>
89          <a href="{{ url_for('faq') }}">FAQ</a>
90          <a href="{{ url_for('contact') }}">Contact</a>
91      </div>
92      <div class="footer-socials">
93          <a href="https://www.instagram.com/miaouff_association/" target="_blank"><i class="fab fa-instagram"></i>
94          <a href="https://www.facebook.com/miaouff_association/" target="_blank"><i class="fab fa-facebook"></i>
95          <a href="https://www.tiktok.com/@miaouff_association/" target="_blank"><i class="fab fa-tiktok"></i></a>
96      <div class="footer-logo">
97          <a href="{{ url_for('home') }}">
99  </footer>
100
101  </body>
102  </html>
103
```

Côté serveur : mise à jour automatique depuis la base de données

Certaines parties du site récupèrent les données directement depuis la base. Par exemple, sur la page **produits**, l'affichage des **catégories** est généré dynamiquement depuis la base de données. Ainsi :

- L'ajout ou la suppression d'une catégorie se reflète immédiatement sur l'interface, sans modification manuelle du HTML.
- Les utilisateurs voient toujours des données à jour.

Capture d'écran de la page produits du site internet :



Capture d'écran du code de **products.html** qui récupère dynamiquement les catégories :

```
6 <div class="filters">
7   <ul>
8     <li><a href="{{ url_for('products') }}" {% if not selected_category %}class="active"{% endif %}>Tous</li>
9     {% for category in categories %}
10      <li>
11        <a href="{{ url_for('products', category=category.name) }}"
12          {% if selected_category == category.name %}class="active"{% endif %}>
13          {{ category.name }}
14        </a>
15      </li>
16    {% endfor %}
17  </ul>
18 </div>
```

Côté client : interactions dynamiques avec JavaScript

Pour améliorer l'expérience utilisateur, j'ai intégré des interactions dynamiques côté client. Par exemple, dans un **mini-jeu quiz** :

- Le message affiché change selon le score du joueur, sans recharger la page.
- L'interface réagit en temps réel aux actions de l'utilisateur, offrant une expérience plus engageante.

Captures d'écran du quizz avec un message de fin différent :

Quiz : Testez vos connaissances sur les animaux !

Que faire si votre chat a des puces ?

Rien

Le laver avec un shampoing anti-puces

Attendre que ça passe

Quiz terminé ! Votre score est de 2 sur 10.

Vous pouvez faire mieux, réessayez ! 😊

Et :

Quiz : Testez vos connaissances sur les animaux !

Quelle est la température corporelle normale d'un chien ?

37°C

38-39°C

40-41°C

Quiz terminé ! Votre score est de 9 sur 10.

Bien joué ! Vous avez un bon score 👍

Extrait du code JavaScript qui permet le changement de message en fonction du score :

```
let message = "";
if (score === selectedQuestions.length) {
    message = "Félicitations ! Vous avez tout juste 🎉";
} else if (score > selectedQuestions.length / 2) {
    message = "Bien joué ! Vous avez un bon score 👍";
} else {
    message = "Vous pouvez faire mieux, réessayez ! 😊";
}
```

Respect de l'accessibilité et de l'UX

- Tous les éléments dynamiques sont accessibles via le clavier et compatibles lecteurs d'écran.
- Les couleurs et contrastes respectent les normes pour les personnes daltoniennes.

3. Développer des composants métier

Respect des bonnes pratiques de la programmation orientée objet (POO)

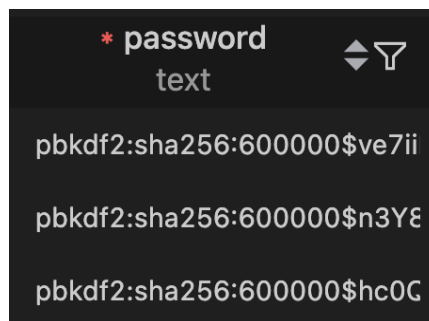
Dans ce projet, la conception des composants métier côté serveur suit rigoureusement les principes de la programmation orientée objet afin d'assurer un code clair, maintenable et sécurisé.

- Encapsulation :
Les données sensibles sont protégées en cachant les attributs critiques et en fournissant des méthodes publiques dédiées pour leur accès contrôlé. Par exemple, l'attribut password de la classe **User** n'est jamais stocké en clair. Il est géré via des méthodes qui utilisent les fonctions de hachage sécurisées de **Werkzeug** (*generate_password_hash* et *check_password_hash*), garantissant que le mot de passe n'est jamais exposé directement dans le code ou en base de données.

Extrait de la route pour créer un nouvel utilisateur par le formulaire d'inscription :

```
hashed_password = generate_password_hash(password, method="pbkdf2:sha256")
new_user = User(email=email, password=hashed_password, role="user")
db.session.add(new_user)
db.session.commit()
```

Les mots de passe des utilisateurs dans la base de données :



A screenshot of a database table with the following content:

* password
pbkdf2:sha256:600000\$ve7ii
pbkdf2:sha256:600000\$n3Y8
pbkdf2:sha256:600000\$hc0G

- Héritage :
La classe **User** hérite de **UserMixin** de la bibliothèque **Flask-Login**, ce qui permet d'intégrer facilement la gestion des sessions et l'authentification des utilisateurs sans avoir à réécrire les fonctionnalités classiques. Ce mécanisme facilite aussi l'extension future des fonctionnalités liées aux utilisateurs.
- Principe de responsabilité unique (Single Responsibility Principle) :
Chaque classe est conçue pour gérer une responsabilité métier bien définie. Par exemple :
 - La classe **Shelter** gère uniquement les données et méthodes liées aux refuges.
 - La classe **Animal** s'occupe des espèces et leurs caractéristiques.
 - La classe **Pet** prend en charge les animaux de compagnie spécifiques, liés aux refuges et aux animaux.
 - La classe **Product** contrôle la logique liée aux produits du magasin, notamment la gestion des stocks.



Cette organisation modulaire améliore la lisibilité, facilite la maintenance et permet d'isoler les évolutions fonctionnelles.

- Conventions de nommage :

Le code respecte les conventions PEP8, avec du `snake_case` pour les noms de variables et fonctions, et du `CamelCase` pour les classes. Les noms sont explicites pour refléter clairement leur rôle, par exemple `shelter_id`, `adoption_date` ou `get_password_hash()`.

```
class Pet(db.Model):
    __tablename__ = "pets"
    pet_id = Column(Integer, primary_key=True)
    name = Column(String(255), nullable=False)
    age = Column(Integer, nullable=False)
```

Documentation du code

Chaque classe, méthode et fonction est documentée avec des docstrings conformes à la norme Python, précisant les paramètres, le comportement, les valeurs de retour, ainsi que les exceptions éventuelles.

Des commentaires clairs complètent la documentation, en particulier pour expliquer la logique métier complexe ou les choix techniques importants.

```
100
101 class User(db.Model, UserMixin):
102     """
103     User model representing an application user.
104
105     Attributes:
106         user_id (int): Primary key, unique identifier for the user.
107         first_name (str): User's first name.
108         last_name (str): User's last name.
109         email (str): User's email address, unique and required.
110         password (str): Hashed password of the user, stored securely.
111         address_number (str): User's street number.
112         street_name (str): Name of the street for user's address.
113         address_complement (str): Additional address information.
114         postal_code (str): Postal code of the user's address.
115         city (str): City of the user's address.
116         country (str): Country of the user's address.
117         phone (str): User's phone number.
118         role (str): Role of the user in the system, defaults to "user".
119         birth_date (date): User's date of birth.
120         orders (relationship): Relationship to Order objects linked to the user.
121     """
```

```
122
123     __tablename__ = "users"
124     user_id = Column(Integer, primary_key=True)
125     first_name = Column(String(255))
126     last_name = Column(String(255))
127     email = Column(String(255), unique=True, nullable=False)
128     password = Column(Text, nullable=False) # Stored as a hashed password
129     address_number = Column(String(10))
130     street_name = Column(String(255))
131     address_complement = Column(String(255))
132     postal_code = Column(String(10))
133     city = Column(String(100))
134     country = Column(String(100))
135     phone = Column(String(20))
136     role = Column(String(50), nullable=False, default="user") # User role: 'user' or 'admin'
137     birth_date = Column(Date)
138     orders = relationship("Order", back_populates="user") # One-to-many relationship with orders
139
140     def get_id(self):
141         """
142         Return the unique identifier of the user as a string.
143
144         Returns:
145         | str: The user_id converted to string.
146         """
147         return str(self.user_id)
```

Réalisation des tests unitaires

Des tests unitaires couvrent les fonctionnalités métier clés, notamment :

- La création, modification et suppression des entités (refuges, animaux, animaux de compagnie, produits, utilisateurs).
- La validation des règles métiers (par exemple, vérification qu'une date d'adoption ne soit pas future).
- La gestion des erreurs et exceptions.

Ces tests garantissent que chaque composant fonctionne comme attendu et facilitent la détection précoce de régressions.

```
tests > test_login.py > ...
1  import sys
2  import os
3
4  sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))
5
6  from app import app, db, User
7  from werkzeug.security import generate_password_hash
8
9
10 def setup_test_user():
11     with app.app_context():
12         user = User.query.filter_by(email="test@example.com").first()
13         if not user:
14             user = User(
15                 email="test@example.com",
16                 password=generate_password_hash("Password123!", method="pbkdf2:sha256"),
17             )
18             db.session.add(user)
19             db.session.commit()
20
```

```
21
22 def test_login_valid_credentials():
23     setup_test_user()
24     with app.test_client() as client:
25         response = client.post(
26             "/login",
27             data=dict(
28                 email="test@example.com", password="Password123!", action="login"
29             ),
30             follow_redirects=True,
31         )
32         assert b"account" in response.data or response.status_code == 200
33
34
35 def test_login_invalid_credentials():
36     with app.test_client() as client:
37         response = client.post(
38             "/login",
39             data=dict(email="wrong@example.com", password="wrongpass", action="login"),
40             follow_redirects=True,
41         )
42         assert b"Identifiants incorrects" in response.data
43
```

Démarche structurée de résolution de problème



En cas de problème détecté pendant les tests ou en production, la démarche suivante est appliquée :

1. Reproduire l'erreur dans un environnement contrôlé pour bien cerner son origine.
2. Analyser les journaux d'erreurs (logs) pour identifier la cause.
3. Tester les composants isolément afin d'isoler la source du dysfonctionnement.
4. Corriger le code en ajoutant, si besoin, des commentaires explicatifs.
5. Relancer les tests unitaires pour vérifier que la correction est efficace et n'a pas introduit de nouvelles erreurs.

Cette méthodologie assure la stabilité, la fiabilité et la qualité continue du projet.

4. Contribuer à la gestion d'un projet informatique

Planification et suivi des tâches

L'outil Notion a été utilisé pour organiser et suivre les tâches du projet selon le principe Kanban. Les tâches ont été découpées par fonctionnalité et par domaine (front-end, back-end, base de données, tests, déploiement), puis planifiées sur une ligne temporelle.

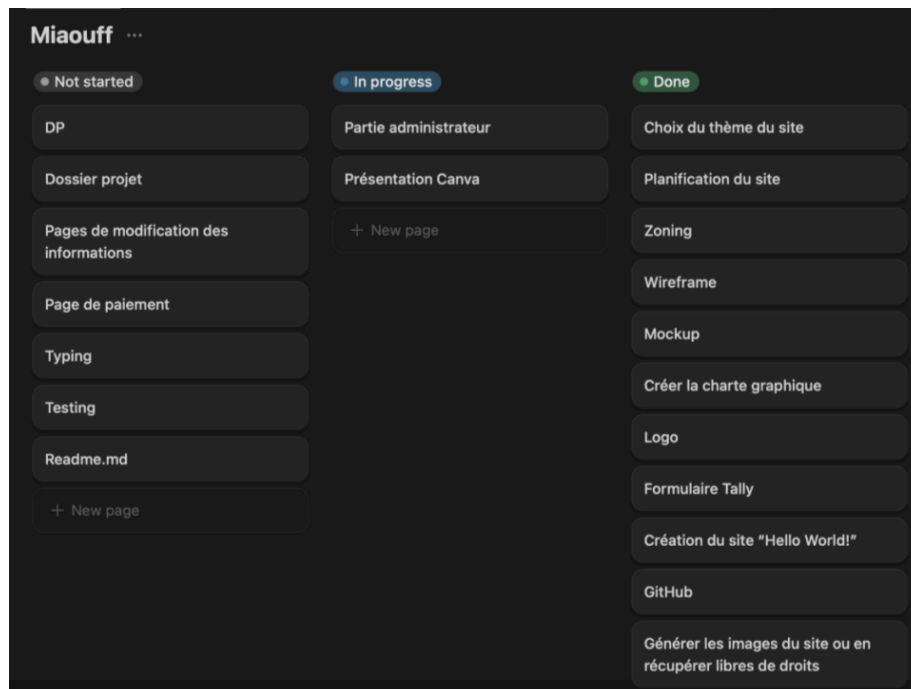
Le suivi régulier a permis d'identifier les éventuels retards et d'ajuster les priorités en conséquence, notamment lorsque certaines fonctionnalités se révélaient plus complexes que prévu (comme la gestion du panier ou l'intégration de MongoDB).

Deux vues complémentaires ont été mises en place :

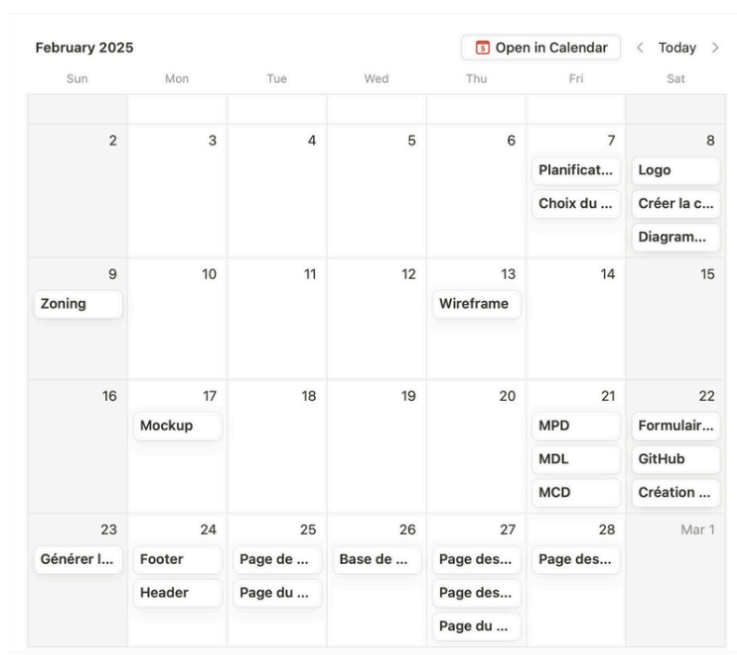
- **Vue Kanban** : classification des tâches par état d'avancement (à faire, en cours, terminé).
- **Vue Calendrier** : planification des tâches sur une ligne temporelle afin de visualiser les échéances.

DOSSIER PROFESSIONNEL (DP)

Vue Kanban :



Vue Calendrier :





Environnement de développement

L'environnement de développement a été défini en adéquation avec l'architecture du projet : Python avec Flask, PostgreSQL et MongoDB pour les bases de données, Git pour le versionnement. Cette configuration a été documentée dans le fichier README.md du dépôt GitHub afin de faciliter la reprise du projet par un autre développeur.

Outils collaboratifs

Dans le cadre de ce projet réalisé en autonomie, les outils suivants ont été mis en place :

- GitHub pour le partage et la sauvegarde du code source
- Notion pour la gestion des tâches et la documentation du projet
- Looping pour la modélisation des bases de données
- Figma pour la création et le partage des maquettes

Procédures qualité

Des procédures qualité ont été mises en œuvre tout au long du projet : revues régulières du code avec Flake8 et Black, tests unitaires et documentation systématique du code via des docstrings et des commentaires.

2. Précisez les moyens utilisés :

Pour la réalisation du site web de Miaouff, j'ai utilisé l'éditeur VS Code et j'ai créé l'application en Python avec le framework Flask.

3. Avec qui avez-vous travaillé ?

Projet réalisé en autonomie

4. Contexte

Nom de l'entreprise, organisme ou association ▶ Cliquez ici pour taper du texte.

Chantier, atelier, service	▶	Projet réalisé lors de la formation
Période d'exercice	▶	Du 29/09/2025 au 14/09/2026

DOSSIER PROFESSIONNEL ^(DP)

5. Informations complémentaires *(facultatif)*

Activité-type 2

Concevoir et développer une application sécurisée organisée en couches

Exemple n°1 - Projet Miaouff

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

1. Analyser les besoins et maquetter une application

Pour concevoir une base de données relationnelle, la première étape consiste à identifier et analyser les besoins des futurs utilisateurs. Cette phase permet ensuite de définir un schéma conceptuel de données adapté.

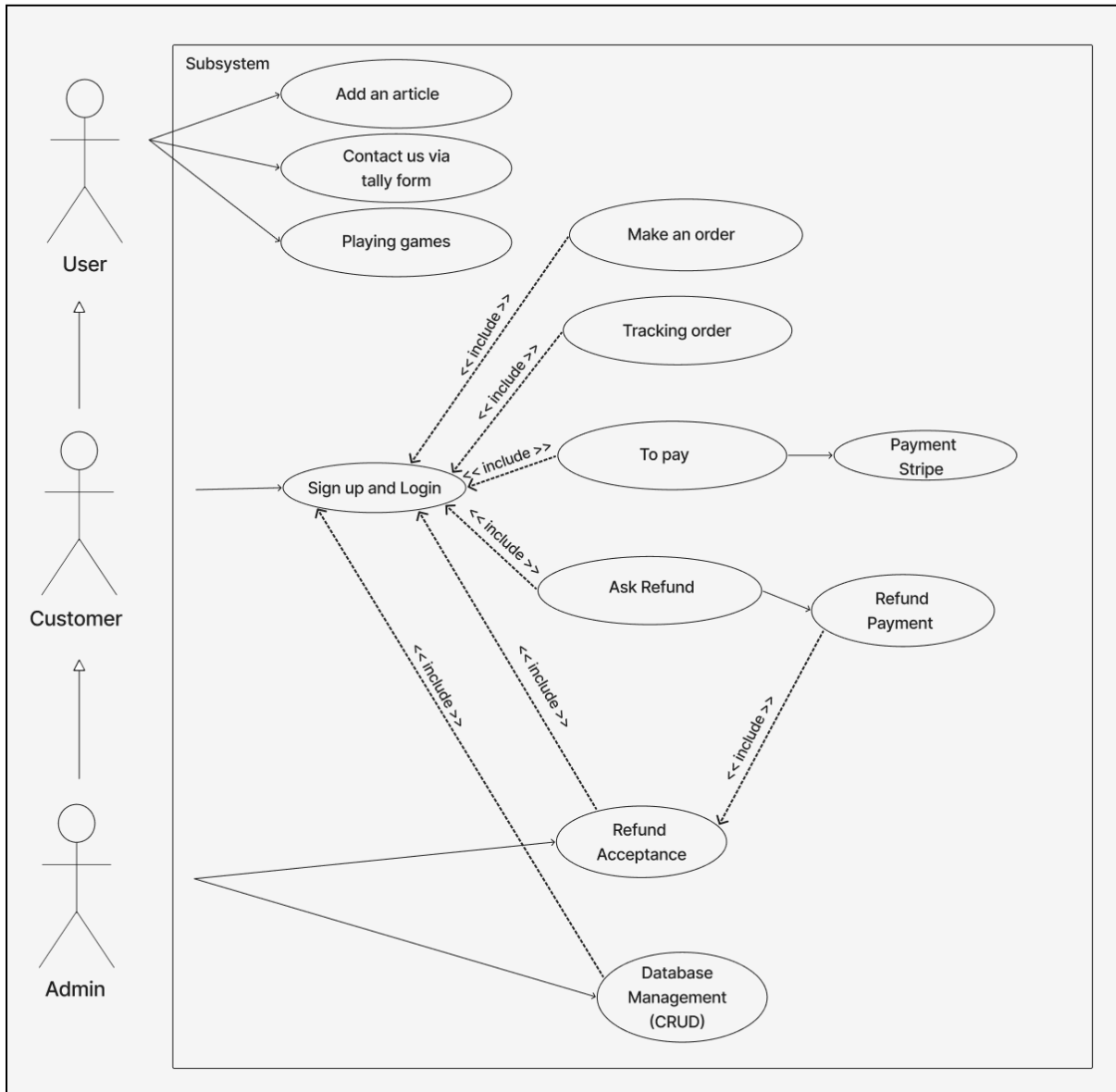
Recueil des besoins utilisateurs :

- Les utilisateurs doivent pouvoir consulter les différents refuges, animaux à adopter ainsi que le glossaire.
- Ils ont la possibilité de créer un compte, de s'authentifier et de se déconnecter.
- Ils peuvent effectuer des achats et suivre leur commande.
- Ils ont la possibilité de jouer à des jeux éducatifs et ludiques.

Il est important de préciser qu'il existe plusieurs types d'acteurs :

- **les utilisateurs**, qui accèdent à la plateforme sans avoir de compte.
- **les clients, qui sont inscrits**, disposant des fonctionnalités complètes.
- **l'administrateur**, chargé de la gestion des comptes utilisateurs et des bases de données.

Diagramme UML des cas d'utilisation (Unified Modeling Language) :

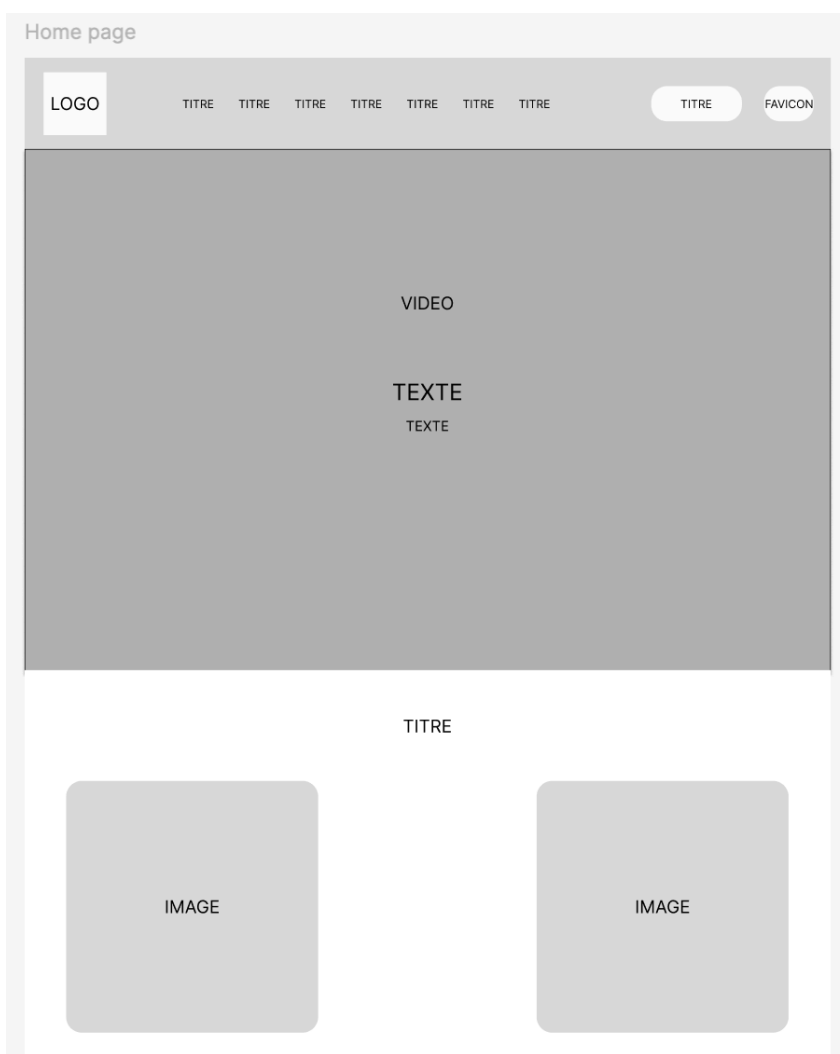


Règles de gestion

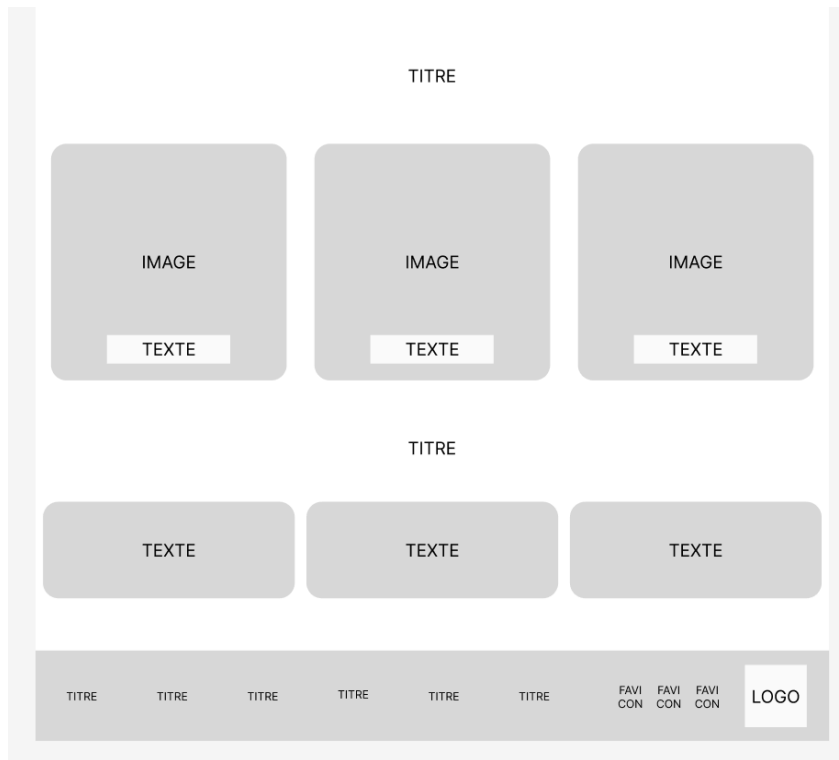
- Chaque animal à adopter doit appartenir à un refuge et à une espèce du glossaire
- Seuls les clients inscrits sont autorisés à effectuer des achats et suivre leurs commandes
- La gestion des contenus est réservée exclusivement aux administrateurs
- Un animal dont la date d'adoption est renseignée est automatiquement masqué de la liste d'adoption

Pour la création des différentes maquettes, j'ai utilisé l'outil Figma.

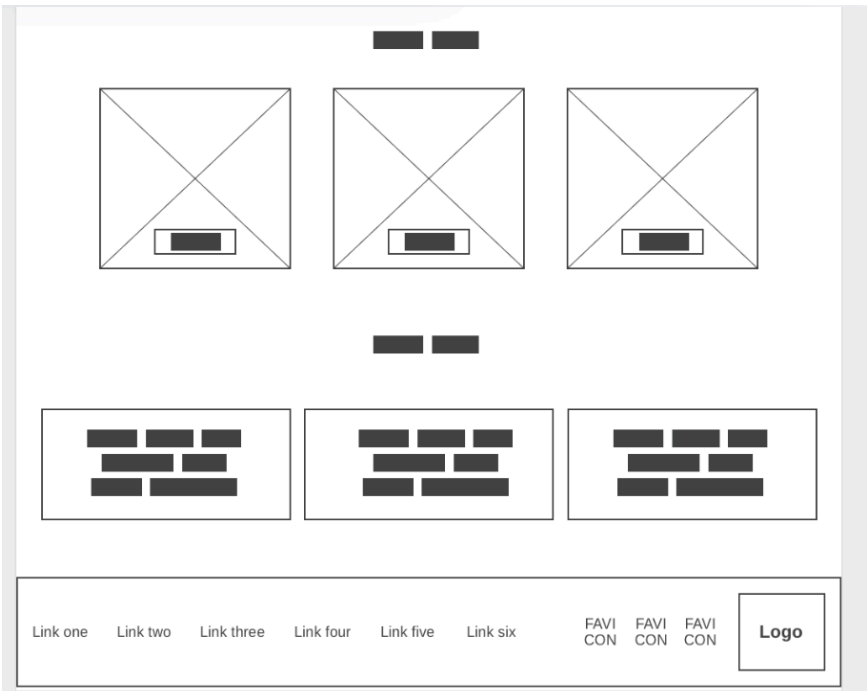
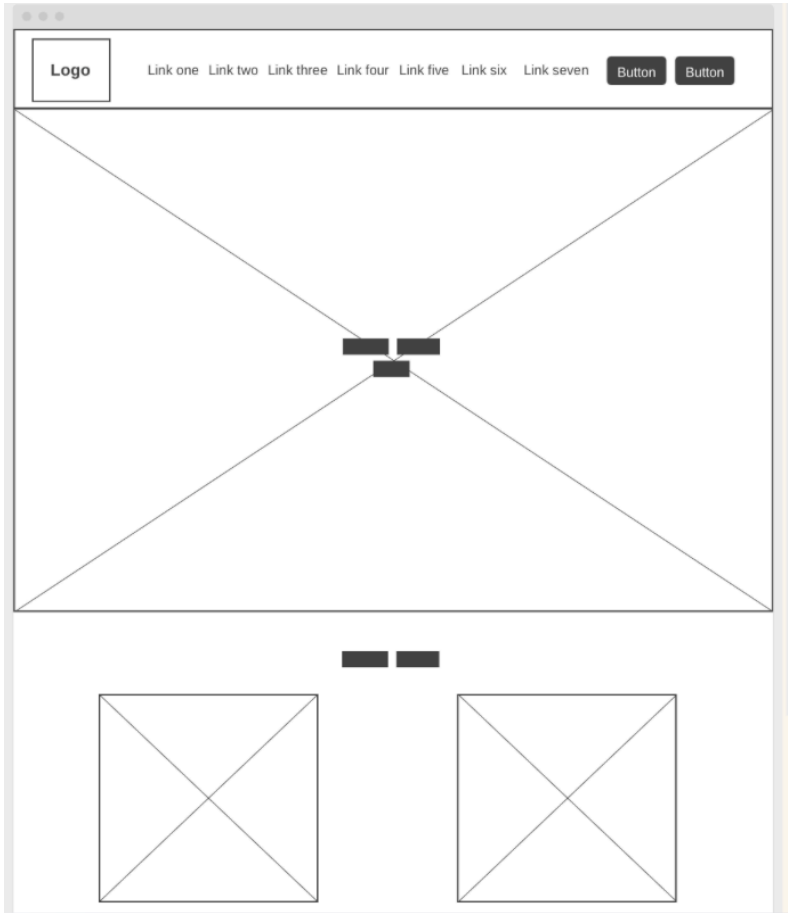
Zoning de la page d'accueil :



DOSSIER PROFESSIONNEL (DP)



Wireframe de la page d'accueil :



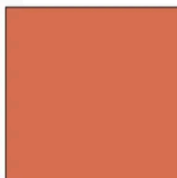


Pour la création de ma charte graphique, j'ai utilisé **Canva**.

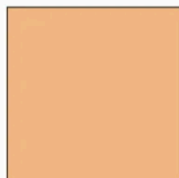


Miaouff

Charte graphique



#e66a4a
RVB(230, 106, 74)
CMJN(0, 54, 68, 10)



#feb47b
RVB(254, 180, 123)
CMJN(0, 29, 52, 0)



#6b8e23
RVB(107, 142, 35)
CMJN(25, 0, 75, 44)



#c4d6a5
RVB(196, 214, 165)
CMJN(8, 0, 23, 16)

Titres : Quicksand (36px)

Texte et paragraphe courant : Inter (18px)

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam sapien leo, tristique vitae orci vitae, vestibulum luctus lorem. Proin at mauris eu augue porttitor ultricies. Mauris nec tempor neque. Proin pharetra, massa eu volutpat facilisis, ante diam finibus tortor, nec iaculis tortor risus quis augue.

Logo

Le logo représente l'identité de l'association Miaouff et donne immédiatement le ton du site : simple, chaleureux et centré sur la cause animale. Il s'intègre facilement sur tous les supports (web, papier, réseaux sociaux).

Couleurs

La palette a été choisie pour transmettre une ambiance douce, accessible et accueillante. Elle repose sur des couleurs sobres et chaleureuses, en accord avec les valeurs de l'association.

Cette combinaison permet une lecture confortable et met en valeur les éléments importants sans surcharger visuellement le site.



Typographie

J'ai choisi la police Quicksand pour les titres et pour le corps Inter, une sans-serif très répandue et facile à lire. Elle permet une bonne accessibilité, notamment pour les personnes dyslexiques ou neuroatypiques, tout en assurant une compatibilité optimale sur tous les navigateurs.

La hiérarchie des textes est claire : titres visibles, paragraphes espacés, et liens bien identifiables. L'objectif est de guider l'œil sans effort, pour que le visiteur puisse se concentrer sur le contenu.

Accessibilité et cohérence visuelle

L'ensemble du site a été pensé pour rester lisible et cohérent, quelle que soit la page. Les couleurs et les typographies sont utilisées de manière constante, et tous les éléments interactifs (boutons, menus, liens) gardent le même style pour faciliter l'expérience utilisateur.

Des vérifications ont été faites pour garantir des contrastes suffisants. Les textes ne sont jamais superposés à des images, les zones cliquables sont assez grandes, et la navigation est fluide. Le site vise à être accessible au plus grand nombre, sans sacrifier l'esthétique ni la clarté.

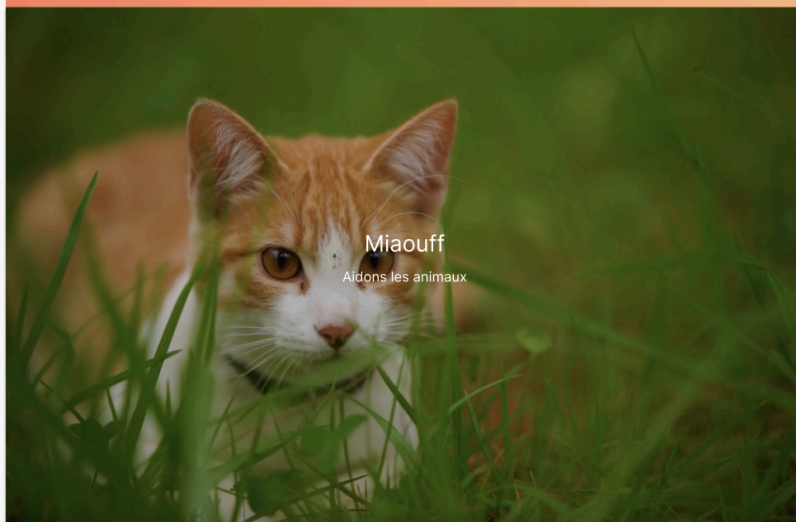
Mockup de la page d'accueil :

Home page



[Accueil](#) [Refuges](#) [Animaux](#) [Produits](#) [Conseils](#) [Blog](#) [Jeux](#)

[Mon compte](#)

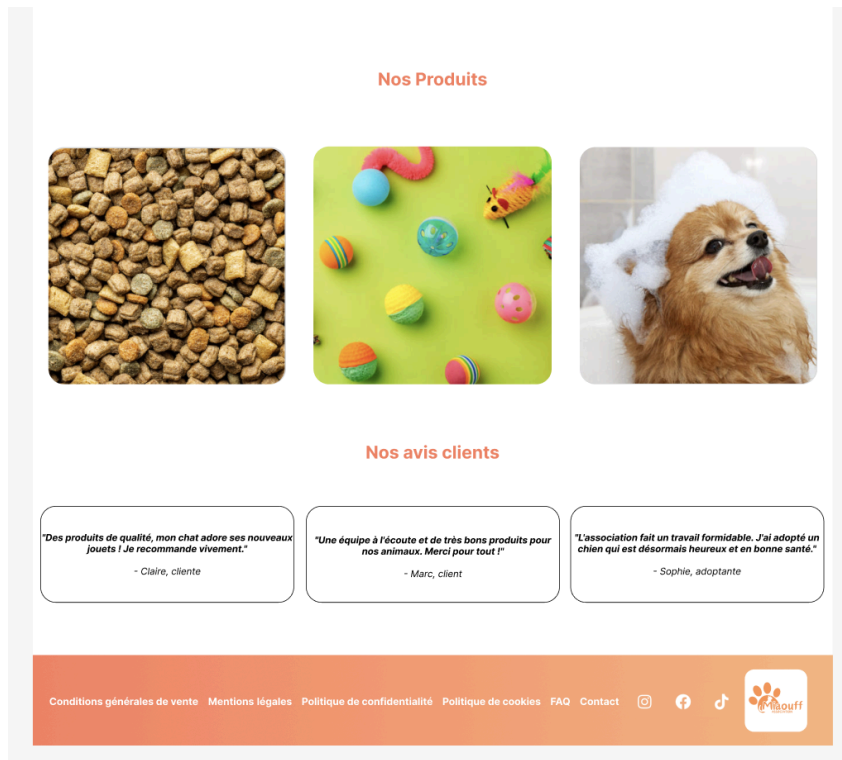


Miaouff

Aidons les animaux

Nos Refuges



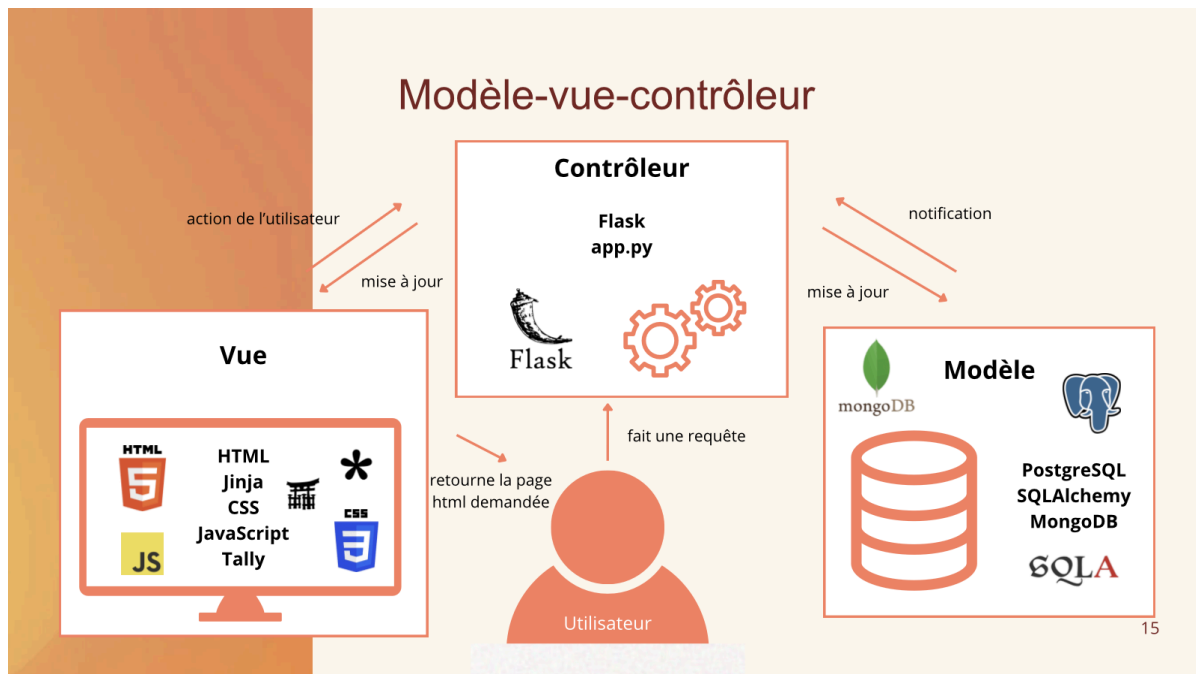


2. Définir l'architecture logicielle d'une application

La structure du projet respecte globalement les principes de l'architecture Modèle–Vue–Contrôleur (MVC), bien que l'organisation du code soit concentrée principalement dans un fichier principal `app.py` qui contient à la fois les routes (contrôleurs) et les interactions avec la base de données.

Un fichier distinct `setup_db.py` a été créé pour initialiser et configurer la base de données.

Réalisation de l'application selon l'architecture modèle-vue-contrôleur :



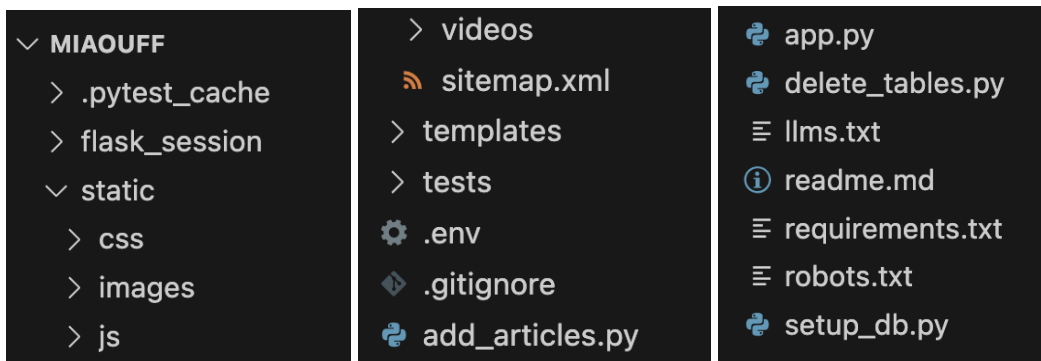
L'architecture modèle-vue-contrôleur (MVC) est composée de trois types de modules :

- un modèle qui définit la structure des données
- une vue qui contient la présentation des données
- un contrôleur qui contient la logique concernant les actions effectuées par l'utilisateur grâce aux routes

Dans l'organisation actuelle :

- Le fichier app.py contient la majorité des routes ainsi que la logique métier (contrôleurs).
- La définition des modèles est intégrée dans le même fichier ou répartie selon les besoins.
- Les templates HTML, correspondant aux vues, sont rangés dans le dossier templates.

- Le dossier static contient les fichiers CSS, les fichiers JavaScript, les images du site, et la vidéo de la page d'accueil.



3. Concevoir et mettre en place une base de données relationnelle

À partir des besoins identifiés auprès des différents acteurs et des cas d'utilisation représentés en diagramme UML, nous pouvons désormais passer à la conception du schéma conceptuel des données. Cette conception suit plusieurs étapes :

- Définition des **règles de gestion**
- Élaboration du **dictionnaire des données**
- Construction du **schéma conceptuel des données** : création des entités, définition des associations, puis ajout des cardinalités.

Règles de gestion :

- Chaque animal à adopter doit appartenir à un refuge et à une espèce du glossaire.
- Seuls les clients inscrits sont autorisés à effectuer des achats et suivre leurs commandes.
- La gestion des comptes utilisateurs et des bases de données est réservée exclusivement aux administrateurs.

Dictionnaire de données :

DOSSIER PROFESSIONNEL (DP)

Entité	Attribut	Description	Type	Contraintes
shelters	shelter_id	Identifiant unique du refuge	N — Numérique	Clé primaire, auto-incrémentée
shelters	name	Nom du refuge	A — Alphabétique	Obligatoire
shelters	address	Adresse complète du refuge	AN — Alphanumérique	Optionnel
shelters	phone	Numéro de téléphone du refuge	AN — Alphanumérique	Format téléphone
shelters	email	Adresse e-mail du refuge	AN — Alphanumérique	Format e-mail valide, max 320 caractères
shelters	description	Description du refuge	AN — Alphanumérique	Optionnel
shelters	image	Nom du fichier image du refuge	AN — Alphanumérique	Optionnel
shelters	user_id	Administrateur ayant les droits de modification	N — Numérique	Clé étrangère → users.user_id
animals	animal_id	Identifiant unique de l'espèce/race	N — Numérique	Clé primaire, auto-incrémentée
animals	species	Espèce (chien, chat...)	A — Alphabétique	Obligatoire
animals	breed	Race (Labrador, Maine Coon...)	A — Alphabétique	Optionnel
animals	lifespan	Durée de vie moyenne	A — Alphabétique	Optionnel
animals	diet	Régime alimentaire	AN — Alphanumérique	Optionnel
animals	specifics	Informations spécifiques (hypoallergénique, etc.)	AN — Alphanumérique	Optionnel
animals	image	Image représentative	AN — Alphanumérique	Optionnel

DOSSIER PROFESSIONNEL (DP)

animals	user_id	Administrateur pouvant modifier	N — Numérique	Clé étrangère → users.user_id
pets	pet_id	Identifiant unique de l'animal	N — Numérique	Clé primaire, auto-incrémentée
pets	name	Nom de l'animal	A — Alphabétique	Obligatoire
pets	age	Âge en années	N — Numérique	≥ 0
pets	gender	Sexe (Male, Female)	A — Alphabétique	Valeurs contrôlées
pets	description	Description de l'animal	AN — Alphanumérique	Optionnel
pets	adoption_date	Date d'adoption (NULL si disponible)	D — Date	Optionnel
pets	shelter_id	Refuge où se trouve l'animal	N — Numérique	Clé étrangère → shelters.shelter_id
pets	animal_id	Correspondance espèce/race	N — Numérique	Clé étrangère → animals.animal_id
pets	image	Nom du fichier image de l'animal	AN — Alphanumérique	Optionnel
pets	user_id	Administrateur pouvant modifier	N — Numérique	Clé étrangère → users.user_id
products	product_id	Identifiant unique du produit	N — Numérique	Clé primaire, auto-incrémentée
products	name	Nom du produit	A — Alphabétique	Obligatoire
products	description	Description du produit	AN — Alphanumérique	Optionnel
products	price_excl_tax	Prix hors taxe	N — Numérique	≥ 0
products	price_incl_tax	Prix TTC	N — Numérique	≥ 0
products	stock	Quantité en stock	N — Numérique	≥ 0

DOSSIER PROFESSIONNEL (DP)

products	category_id	Catégorie du produit	N — Numérique	Clé étrangère → categories.category_id
products	weight	Poids du produit	N — Numérique	Optionnel
products	image	Nom du fichier image du produit	AN — Alphanumériqu e	Optionnel
categories	category_id	Identifiant unique de la catégorie	N — Numérique	Clé primaire, auto-incrémentée
categories	name	Nom de la catégorie	A — Alphabétique	Obligatoire
users	user_id	Identifiant unique de l'utilisateur	N — Numérique	Clé primaire, auto-incrémentée
users	first_name	Prénom de l'utilisateur	A — Alphabétique	Obligatoire
users	last_name	Nom de famille	A — Alphabétique	Obligatoire
users	email	Adresse e-mail	A — Alphabétique	Unique, format e-mail (jusqu'à 320 caractères)
users	password	Mot de passe haché	AN — Alphanumériqu e	Obligatoire
users	birth_date	Date de naissance	D — Date	Optionnel
users	address_nu mber	Numéro d'adresse	AN — Alphanumériqu e	Optionnel
users	street_name	Nom de rue	A — Alphabétique	Optionnel
users	address_co mplement	Complément d'adresse	AN — Alphanumériqu e	Optionnel
users	postal_code	Code postal	AN — Alphanumériqu e	Optionnel
users	city	Ville	A — Alphabétique	Optionnel
users	country	Pays	A —	Optionnel

DOSSIER PROFESSIONNEL (DP)

			Alphabétique	
users	phone	Téléphone	AN — Alphanumérique	Optionnel
users	role	Rôle (customer, admin...)	A — Alphabétique	Obligatoire
orders	order_id	Identifiant unique de la commande	N — Numérique	Clé primaire, auto-incrémentée
orders	user_id	Client ayant passé la commande	N — Numérique	Clé étrangère → users.user_id
orders	order_date	Date et heure de commande	H — Date/Heure	Obligatoire
orders	status	Statut (pending, shipped...)	A — Alphabétique	Obligatoire
orders	total_price_excl_tax	Total HT	N — Numérique	≥ 0
orders	total_price_incl_tax	Total TTC	N — Numérique	≥ 0
orders	shipping_fee	Frais de livraison	N — Numérique	≥ 0
order_products	order_id	Commande associée	N — Numérique	Clé étrangère → orders.order_id
order_products	product_id	Produit associé	N — Numérique	Clé étrangère → products.product_id
order_products	quantity	Quantité commandée	N — Numérique	≥ 1
order_products	unit_price_excl_tax	Prix unitaire HT	N — Numérique	≥ 0
order_products	unit_price_incl_tax	Prix unitaire TTC	N — Numérique	≥ 0
payments	payment_id	Identifiant unique du paiement	N — Numérique	Clé primaire, auto-incrémentée
payments	payment_method	Méthode de paiement	A — Alphabétique	Obligatoire
payments	payment_status	Statut du paiement	A —	Obligatoire

DOSSIER PROFESSIONNEL (DP)



	atus		Alphabétique	
payments	payment_date	Date et heure du paiement	H — Date/Heure	Obligatoire
payments	payment_total	Montant payé	N — Numérique	≥ 0
payments	order_id	Commande liée	N — Numérique	Clé étrangère → orders.order_id

Entités :

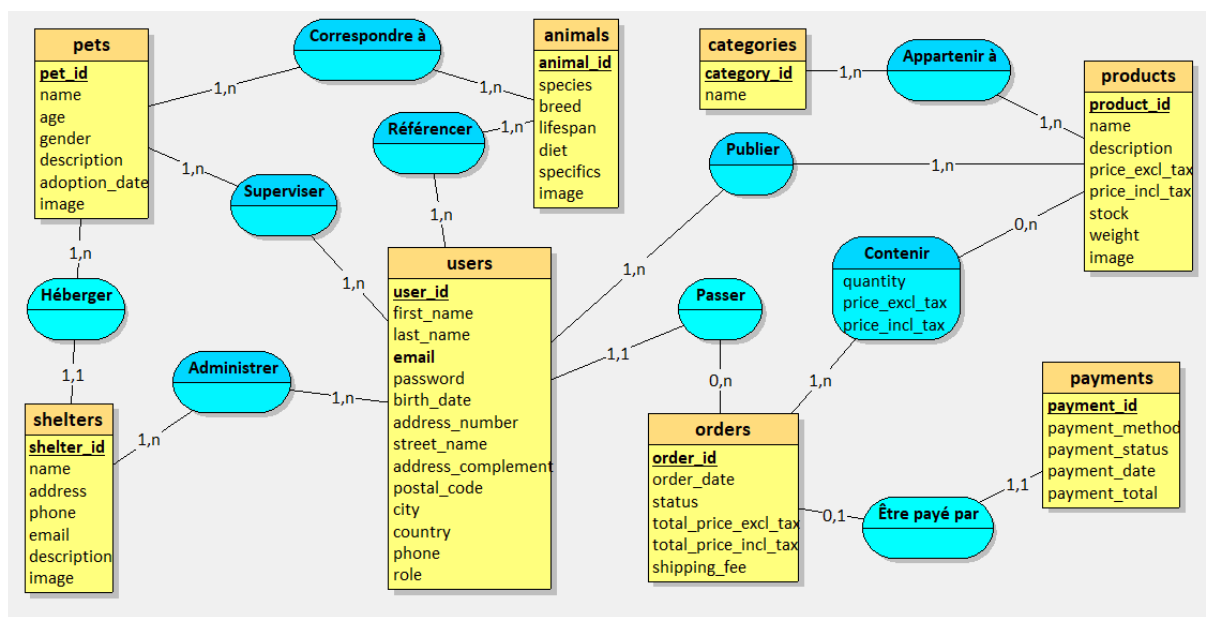
- **Refuge (shelters)** : informations sur les refuges partenaires (nom, adresse, contact, description, image).
- **Espèce / Race (animals)** : caractéristiques générales des animaux (espèce, race, espérance de vie, alimentation, spécificités, image).
- **Animal à adopter (pets)** : animaux présents dans les refuges, disponibles ou déjà adoptés.
- **Produit (products)** : articles proposés à la vente (description, prix, stock, catégorie, image).
- **Catégorie (categories)** : classification des produits de la boutique.
- **Utilisateur (users)** : informations relatives aux clients et administrateurs (données personnelles, rôle, coordonnées).
- **Commande (orders)** : commandes passées par les utilisateurs avec date, statut et montant.
- **Détail de commande (order_products)** : liste des produits associés à chaque commande avec quantité et prix unitaire.
- **Paiement (payments)** : informations sur les paiements effectués (méthode, statut, montant, date).

Associations entre les entités :

- Un **refuge** héberge 1 à plusieurs **animaux à adopter**, mais chaque animal appartient à un seul refuge.
- Une **espèce / race** peut correspondre à 1 ou à plusieurs **animaux à adopter**, mais chaque animal est associé à une seule espèce.
- Un **utilisateur** peut créer un compte et effectuer 0 à plusieurs **commandes**.
- Une **commande** est toujours liée à un seul **utilisateur**, mais peut contenir 1 ou plusieurs **produits** via l'entité **détail de commande**.
- Un **produit** appartient obligatoirement à une ou plusieurs **catégories**, et une catégorie peut regrouper plusieurs produits.
- Un **paiement** est obligatoirement lié à une seule **commande**.
- Seuls les **administrateurs** peuvent gérer les informations relatives aux refuges, aux espèces, aux animaux, aux produits et aux utilisateurs.

Les schémas du modèle Merise ont été conçus avec l'outil Looping.

Modèle conceptuel de données (MCD) :



- Les **entités** identifiées dans le modèle conceptuel deviennent des **tables** dans la base de données.
- Les **identifiants numériques** sont traduits en **clés primaires**.
- Les **cardinalités** définies dans le modèle conceptuel sont converties en **clés étrangères** pour gérer les relations « One-to-Many » ou « One-to-One ».
- Lorsque la relation est de type « Many-to-Many », il est nécessaire de créer des **tables intermédiaires** afin d'assurer l'intégrité et la cohérence des données.

```

    erDiagram
        pets ||--o{ animals : Correspondre_à
        Référencer ||--o{ animals
        Superviser ||--o{ pets
        Administrer ||--o{ shelters
        Publier ||--o{ products
        Contenir ||--o{ products
        Appartenir_à ||--o{ categories

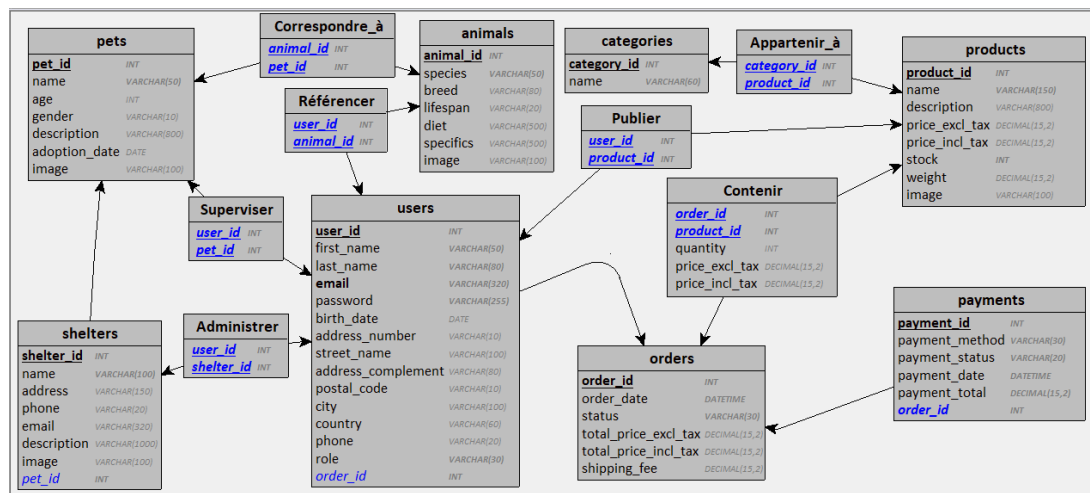
        pets {
            string pet_id PK
            string name
            int age
            string gender
            string description
            datetime adoption_date
            string image
        }
        shelters {
            string shelter_id PK
            string name
            string address
            string phone
            string email
            string description
            string image
            string pet_id FK
        }
        animals {
            string animal_id PK
            string species
            string breed
            int lifespan
            string diet
            string specifics
            string image
        }
        users {
            string user_id PK
            string first_name
            string last_name
            string email
            string password
            datetime birth_date
            int address_number
            string street_name
            string address_complement
            string postal_code
            string city
            string country
            string phone
            string role
            string order_id FK
        }
        categories {
            string category_id PK
            string name
        }
        products {
            string product_id PK
            string name
            string description
            float price_excl_tax
            float price_incl_tax
            int stock
            int weight
            string image
        }
        payments {
            string payment_id PK
            string payment_method
            string payment_status
            datetime payment_date
            float payment_total
            string order_id FK
        }
  
```

Page

La conception du **modèle logique de données** a également été réalisée avec l'outil **Draw.io**.

Une fois ce modèle établi, il est alors possible de passer à l'étape suivante : la conception du **modèle physique de données**, qui reprend la structure définie dans le MLD pour aboutir à une implémentation concrète dans le système de gestion de base de données choisi.

Modèle physique de données (MPD) :



Sécurité, confidentialité et intégrité des données

1. Sécurité des connexions

Dans le cadre de ce projet, les bases de données utilisées sont **PostgreSQL** (via **SQLAlchemy**) et **MongoDB Atlas**.

Même si le site n'est pas encore déployé, la configuration prévue s'appuie sur l'utilisation de protocoles de chiffrement **SSL/TLS** (« Secure Socket Layer » et « Transport Layer Security ») afin de garantir la confidentialité des échanges et de n'autoriser que des connexions sécurisées.

2. Sécurité des accès à la base de données

Les identifiants et informations sensibles de connexion aux bases de données ne sont pas inscrits en clair dans le code source. Ils sont stockés dans un fichier **.env**, chargé grâce à



la bibliothèque **python-dotenv**, ce qui permet de respecter les bonnes pratiques de sécurité et de préserver la confidentialité en cas de partage du projet.

3. Sécurisation des chargements de fichiers

Le projet permet l'ajout d'images (par exemple pour les refuges, les animaux, les animaux à adopter ou les produits). Pour sécuriser ces téléchargements, plusieurs mesures sont mises en place :

- Limitation des types de fichiers autorisés aux seules images (.png, .jpg, .jpeg, .gif).
- Nettoyage systématique des noms de fichiers avec la fonction **secure_filename** afin de prévenir les attaques par chemins ou noms de fichiers piégés.
- Limitation de la taille maximale des fichiers téléchargés pour se protéger contre les attaques par surcharge (DoS).

4. Contrôle des injections et validation des données

L'utilisation de **SQLAlchemy** avec PostgreSQL offre une protection contre les **injections SQL**, car les requêtes sont générées automatiquement plutôt que rédigées en SQL brut. Cependant, une validation stricte des entrées utilisateur est également effectuée afin de garantir l'intégrité des données et d'éviter des failles potentielles. Les permissions au niveau des bases de données sont limitées au strict nécessaire pour réduire les risques.

5. Gestion des sessions et des utilisateurs

L'authentification des utilisateurs est gérée par la bibliothèque **Flask-Login**, qui permet de sécuriser les sessions. Les mots de passe sont systématiquement **hachés** avant leur stockage, afin de garantir leur confidentialité et de réduire les risques en cas de fuite de données.

Mise en place d'une base de données test :

Table **users** :

DOSSIER PROFESSIONNEL (DP)

SELECT * FROM users LIMIT 100

	user_id integer	first_name varchar(255)	last_name varchar(255)	* email varchar(255)	* password text	address_number varchar(10)	street_name varchar(255)
>	1	(NULL)	(NULL)	julie@miaouff.fr	pbkdf2:sha256:600000\$ve7ii	(NULL)	(NULL)
>	2	(NULL)	(NULL)	juju@juju.fr	pbkdf2:sha256:600000\$n3Y€	(NULL)	(NULL)

Table **pets** :

SELECT * FROM pets LIMIT 100

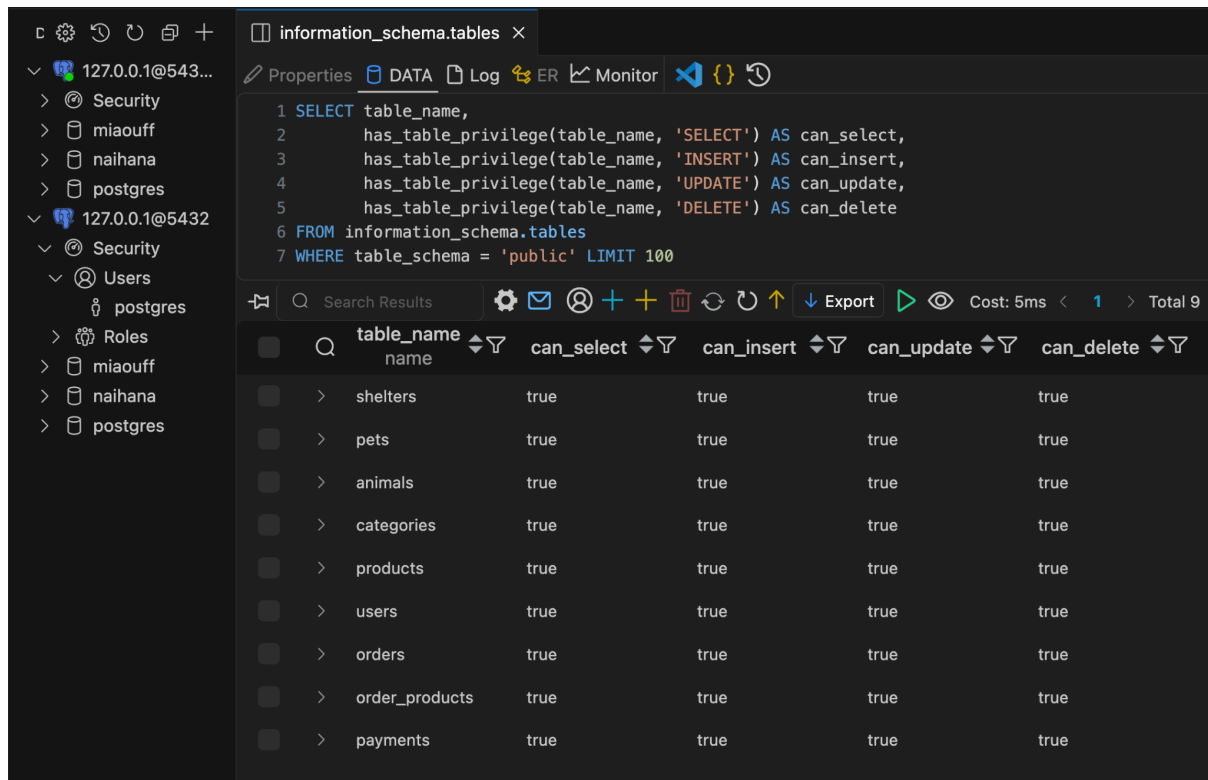
	pet_id integer	* name varchar(255)	* age integer	* gender varchar(10)	description text	adoption_date date	image varchar(200)
>	1	Mew	2	Mâle	Un adorable chat joueur et cur	(NULL)	images/mew.png
>	2	Minou	4	Mâle	Un chat doux et calme qui aime	(NULL)	images/minou.png
>	3	Chippie	1	Femelle	Petite boule d'énergie, Chippie	(NULL)	images/chippie.png
>	4	Max	3	Mâle	Max est un chien affectueux et	(NULL)	images/max.png
>	5	Tonnerre	5	Mâle	Un chien énergique et joueur	(NULL)	images/tonnerre.png
>	6	Daisy	2	Femelle	Daisy est une chienne douce et	(NULL)	images/daisy.png

Droits d'accès aux bases de données :

L'accès aux bases de données **PostgreSQL** et **MongoDB Atlas** est restreint à l'administrateur du projet.

Pour **PostgreSQL**, l'utilisateur principal configuré pour le projet dispose des permissions nécessaires pour créer, lire, mettre à jour et supprimer les tables via **SQLAlchemy**.

Pour **MongoDB Atlas**, un utilisateur spécifique est configuré avec un mot de passe stocké dans le fichier **.env**. Cet utilisateur possède uniquement les droits nécessaires pour gérer les collections liées au blog et aux autres données **MongoDB**.



The screenshot shows a database management interface. On the left is a sidebar with a tree view containing nodes for '127.0.0.1@5432...', 'Security', 'miaouff', 'naihana', 'postgres', and '127.0.0.1@5432'. The 'postgres' node is expanded, showing 'Roles' and 'postgres'. The main area displays a SQL query in the 'DATA' tab:

```
1 SELECT table_name,  
2       has_table_privilege(table_name, 'SELECT') AS can_select,  
3       has_table_privilege(table_name, 'INSERT') AS can_insert,  
4       has_table_privilege(table_name, 'UPDATE') AS can_update,  
5       has_table_privilege(table_name, 'DELETE') AS can_delete  
6 FROM information_schema.tables  
7 WHERE table_schema = 'public' LIMIT 100
```

Below the query, the results are shown in a table with columns: 'table_name', 'can_select', 'can_insert', 'can_update', and 'can_delete'. The results list 9 tables: shelters, pets, animals, categories, products, users, orders, order_products, and payments, all with 'true' values for all privilege columns.

table_name	can_select	can_insert	can_update	can_delete
shelters	true	true	true	true
pets	true	true	true	true
animals	true	true	true	true
categories	true	true	true	true
products	true	true	true	true
users	true	true	true	true
orders	true	true	true	true
order_products	true	true	true	true
payments	true	true	true	true

En environnement de développement, l'administrateur dispose de tous les droits sur les bases de données **PostgreSQL** et **MongoDB Atlas**, ce qui lui permet de créer, modifier et supprimer les tables ou collections nécessaires au projet.

Pour **MongoDB Atlas**, les rôles prédéfinis (**read**, **readWrite**) sont utilisés, et des rôles personnalisés peuvent être créés si besoin.

Sauvegarde des bases de données :

Des sauvegardes peuvent être effectuées localement à l'aide de **pg_dump** pour **PostgreSQL** et **mongodump** pour **MongoDB**, afin de sécuriser les données pendant le développement.

4. Développer des composants d'accès aux données SQL et NoSQL

Structuration des accès aux données : consultation, modification, création et suppression

Dans mon projet, la gestion des bases de données SQL et NoSQL est organisée de manière claire :

- Le fichier **setup_db.py** contient la définition des modèles orientés objet (SQLAlchemy) pour la base de données PostgreSQL.
- Le fichier **add_articles.py** est dédié à la configuration et à l'insertion des données dans la base de données MongoDB utilisée pour la section blog.
- Enfin, les opérations CRUD (Create, Read, Update, Delete) sont principalement gérées dans **app.py**, à travers les routes Flask qui communiquent avec ces bases de données.

Cette organisation permet de séparer la définition des modèles et la configuration des bases de données, tout en centralisant la logique applicative dans le fichier principal de l'application.

🔗 setup_db.py > ...

```
51
52 class Animal(db.Model):
53     __tablename__ = "animals"
54     animal_id = Column(Integer, primary_key=True)
55     species = Column(String(100), nullable=False)
56     breed = Column(String(100), nullable=False)
57     lifespan = Column(String(50))
58     diet = Column(Text)
59     specifics = Column(Text)
60     image = Column(String(200))
61     pets = relationship("Pet", back_populates="animal")
62
```

```
add_articles.py > ...
1  from pymongo import MongoClient
2  from datetime import datetime
3  import os
4  from dotenv import load_dotenv
5
6
7  load_dotenv()
8  mongo_uri = os.getenv("MONGODB_URI")
9  client = MongoClient(mongo_uri)
10 db = client["miaouff_database"]
11 articles_collection = db["miaouff_collection"]
12
13 articles = [
14     {
15         "title": "Adoption de Mia au Havre des Ronronnements",
16         "content": (
17             "Nous avons une merveilleuse nouvelle à partager !",
18             "Après avoir été recueillie, soignée et choyée par",
19             "dans son nouveau foyer, entourée de beaucoup d'amour",
20             " Venez rencontrer nos autres petits pensionnaires à",
21         ),
22         "image_url": "images/mia.png",
23         "shelter_id": 1,
24         "created_at": datetime.now(),
25     },
26
27     {
28         "title": "Rénovation des refuges : Venez découvrir",
29         "content": (
30             "Nous sommes heureux d'annoncer que les deux",
31             "Ces travaux ont permis d'améliorer les conditions",
32             "Nous invitons tous les amoureux des animaux à",
33             "Grâce à votre soutien, nous continuons d'offrir",
34         ),
35         "image_url": "images/renovation-refuge.png",
36         "shelter_id": None,
37         "created_at": datetime.now(),
38     },
39 ]
40
41 articles_collection.insert_many(articles)
42
43 print("Les articles ont été ajoutés avec succès !")
44
45
```

CRUD :

CREATE & UPDATE :

```
@app.route("/edit-article/<article_id>", methods=["GET", "POST"])
@login_required
def edit_article(article_id):
    if current_user.role != "admin":
        flash("Accès refusé.", "danger")
        return redirect(url_for("home"))

    collection = mongo_db.miaouff_collection
    article = collection.find_one({"_id": ObjectId(article_id)})

    if request.method == "POST":
        title = request.form["title"]
        content = request.form["content"]
        shelter_id = request.form.get("shelter_id") or None
        image = request.files.get("image")

        update_data = {
            "title": title,
            "content": content,
            "shelter_id": int(shelter_id) if shelter_id else None,
        }

        if image and image.filename != "":
            image_filename = secure_filename(image.filename)
            image.save(os.path.join(app.config["UPLOAD_FOLDER"], image_filename))
            update_data["image_url"] = f"images/{image_filename}"

        collection.update_one({"_id": ObjectId(article_id)}, {"$set": update_data})
        flash("Article mis à jour", "success")
        return redirect(url_for("edit_articles"))

    shelters = Shelter.query.all()
    return render_template("edit_article.html", article=article, shelters=shelters)
```

READ :

```
@app.route("/blog")
def blog():
    collection = mongo_db.miaouff_collection
    articles = collection.find()
    return render_template("blog.html", articles=articles)
```



DELETE :

```
@app.route("/delete-article/<article_id>", methods=["POST"])
@login_required
def delete_article(article_id):
    if current_user.role != "admin":
        flash("Accès refusé.", "danger")
        return redirect(url_for("home"))

    collection = mongo_db.miaouff_collection
    collection.delete_one({"_id": ObjectId(article_id)})
    flash("Article supprimé", "success")
    return redirect(url_for("edit_articles"))
```

Gestion de l'intégrité des données et des conflits d'accès

Dans la base de données relationnelle PostgreSQL, l'intégrité des données est assurée grâce à l'utilisation de **clés étrangères**. Ces dernières permettent d'établir des relations explicites entre les tables et de garantir la cohérence des informations.

Par exemple :

- Dans la table **pets**, la colonne *shelter_id* fait référence à la table **shelters**. Cela signifie qu'un animal de compagnie ne peut être associé qu'à un refuge existant. Si l'on tente d'insérer un *shelter_id* inexistant, la base de données rejettera automatiquement l'opération.
- De même, la colonne *animal_id* dans **pets** établit une relation avec la table **animals**, garantissant que chaque animal de compagnie correspond bien à une espèce définie.

```
class Pet(db.Model):
    __tablename__ = "pets"
    pet_id = Column(Integer, primary_key=True)
    name = Column(String(255), nullable=False)
    age = Column(Integer, nullable=False)
    gender = Column(String(10), nullable=False)
    description = Column(Text)
    adoption_date = Column(Date, nullable=True)
    image = Column(String(200))
    shelter_id = Column(Integer, ForeignKey("shelters.shelter_id"))
    animal_id = Column(Integer, ForeignKey("animals.animal_id"))
    shelter = relationship("Shelter", back_populates="pets")
    animal = relationship("Animal", back_populates="pets")
```

Concernant la base NoSQL MongoDB (utilisée pour la section blog), les relations ne sont pas gérées par des clés étrangères. L'intégrité repose donc sur la logique applicative, c'est-à-dire sur les contrôles effectués directement dans le code Flask avant insertion ou modification des documents.

Validation et contrôle des entrées côté serveur

Avant l'enregistrement d'un nouvel utilisateur dans la base de données, les données envoyées par le formulaire sont **contrôlées côté serveur**.

Cela permet d'éviter d'insérer des informations incorrectes ou incohérentes et de garantir l'intégrité des données.

Exemple concret : validation d'un email unique

Lors de l'inscription d'un utilisateur, une vérification est effectuée pour s'assurer que l'adresse e-mail n'est pas déjà utilisée.

Si l'email existe déjà dans la table users, l'inscription est bloquée et un message d'erreur est renvoyé à l'utilisateur.

Extrait de la route `/login` dans **app.py** :

```
if action == "register":
    confirm_password = request.form.get("confirm_password")

    if User.query.filter_by(email=email).first():
        flash("Cet e-mail est déjà utilisé.", "danger")
        return redirect(url_for("login"))
```

Cela garantit que chaque compte est associé à une adresse unique, ce qui évite les doublons et renforce la cohérence des données dans la base.

2. Précisez les moyens utilisés :

Pour la charte graphique, j'ai utilisé le site Canva. Pour les maquettes, j'ai utilisé Figma. Enfin, pour la réalisation du site web de Miaouff, j'ai utilisé l'éditeur VS Code et j'ai créé l'application en Python avec le framework Flask.

3. Avec qui avez-vous travaillé ?

Projet réalisé en autonomie

4. Contexte

Nom de l'entreprise, organisme ou association - Cliquez ici pour taper du texte.

Chantier, atelier, service	-	Projet réalisé lors de la formation
Période d'exercice	-	Du 29/09/2025 au 14/09/2026

5. Informations complémentaires (facultatif)

DOSSIER PROFESSIONNEL ^(DP)

Activité-type 3 Préparer le déploiement d'une application sécurisée

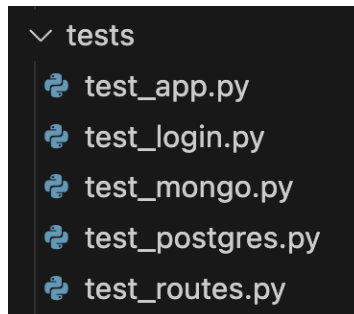
Exemple n°1 - Projet Miaouff

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

1. Préparer et exécuter les plans de tests d'une application

Tests unitaires et de sécurité

L'application intègre une série de tests unitaires afin de garantir le bon fonctionnement des principales fonctionnalités, aussi bien côté SQL (PostgreSQL) que NoSQL (MongoDB). Ces tests sont regroupés dans un dossier *tests/* qui contient plusieurs fichiers (**test_app.py**, **test_login.py**, **test_routes.py**, **test_mongo.py**, **test_postgres.py**).



Chaque fichier se concentre sur un aspect spécifique de l'application :

- **test_app.py** vérifie que la page d'accueil (/) est bien accessible.
- **test_login.py** vérifie le processus de connexion, à la fois avec des identifiants valides et invalides.
- **test_routes.py** teste certaines routes spécifiques, comme le glossaire filtré par espèce.
- **test_mongo.py** s'assure que la connexion à la base de données MongoDB est fonctionnelle et que des documents peuvent être récupérés.

- **test_postgres.py** vérifie la connexion à PostgreSQL et la récupération d'objets (exemple : les catégories).

Capture d'écran du code de **test_login.py** :

```
tests > test_login.py > ...
1  import sys
2  import os
3
4  sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))
5
6  from app import app, db, User
7  from werkzeug.security import generate_password_hash
8
9
10 def setup_test_user():
11     with app.app_context():
12         user = User.query.filter_by(email="test@example.com").first()
13         if not user:
14             user = User(
15                 email="test@example.com",
16                 password=generate_password_hash("Password123!", method="pbkdf2:sha256"),
17             )
18             db.session.add(user)
19             db.session.commit()
20
```

```
21
22 def test_login_valid_credentials():
23     setup_test_user()
24     with app.test_client() as client:
25         response = client.post(
26             "/login",
27             data=dict(
28                 email="test@example.com", password="Password123!", action="login"
29             ),
30             follow_redirects=True,
31         )
32         assert b"account" in response.data or response.status_code == 200
33
34
35 def test_login_invalid_credentials():
36     with app.test_client() as client:
37         response = client.post(
38             "/login",
39             data=dict(email="wrong@example.com", password="wrongpass", action="login"),
40             follow_redirects=True,
41         )
42         assert b"Identifiants incorrects" in response.data
43
```



Ces tests sont exécutés avec **pytest**. Lors du lancement des tests, un rapport s'affiche indiquant le succès ou l'échec de chaque scénario testé.

```
● → miaouff git:(main) x pytest .
===== test session starts =====
platform darwin -- Python 3.9.6, pytest-8.3.5, pluggy-1.5.0
rootdir: /Users/julie/Documents/École/Projets RNCP/Projet Miaouff/Sauvegarde des projets/miaouff
collected 7 items

tests/test_app.py . [ 14%]
tests/test_login.py .. [ 42%]
tests/test_mongo.py . [ 57%]
tests/test_postgres.py . [ 71%]
tests/test_routes.py .. [100%]

===== warnings summary =====
../../../../Library/Python/3.9/lib/python/site-packages/urllib3/_init_.py:35
/Users/julie/Library/Python/3.9/lib/python/site-packages/urllib3/_init_.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
warnings.warn(

../../../../Library/Python/3.9/lib/python/site-packages/flask_session/filesystem/filesystem.py:75
/Users/julie/Library/Python/3.9/lib/python/site-packages/flask_session/filesystem/filesystem.py:75: DeprecationWarning: FileSystemSessionInterface is deprecated and will be removed in a future release. Instead use the CacheLib backend directly.
warnings.warn(

tests/test_login.py::test_login_valid_credentials
/Users/julie/Documents/École/Projets RNCP/Projet Miaouff/Sauvegarde des projets/miaouff/app.py:95: LegacyAPIWarning: The Query.get() method is considered legacy as of the 1.x series of SQLAlchemy and becomes a legacy construct in 2.0. The method is now available as Session.get() (deprecated since: 2.0) (Background on SQLAlchemy 2.0 at: https://sqlalche.me/e/b8d9)
    return User.query.get(int(user_id))

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 7 passed, 3 warnings in 1.43s =====
```

Gestion des erreurs

En cas de dysfonctionnement, l'application s'appuie sur le système de logging de Flask. Les logs permettent de consulter rapidement la nature de l'erreur et d'identifier si elle provient de la logique métier, de la base de données ou d'un problème réseau. Une fois la source identifiée, un nouveau test peut être écrit pour reproduire le scénario problématique et valider la correction apportée.

2. Préparer et documenter le déploiement d'une application

Le projet *Miaouff* n'a été déployé via railway : <https://miaouff.up.railway.app/> . Une procédure de déploiement a été rédigée pour cette étape essentielle. Cette procédure prend en compte les dépendances du projet, les versions des bibliothèques utilisées, ainsi que les bonnes pratiques pour un déploiement sécurisé et évolutif.

Le projet repose sur un environnement Python avec Flask, ainsi qu'une base de données PostgreSQL connectée via SQLAlchemy et une base MongoDB Atlas pour la partie blog. La préparation du déploiement inclut notamment :

- La génération d'un fichier requirements.txt listant toutes les dépendances nécessaires au fonctionnement de l'application.
- L'intégration du serveur WSGI (Web Server Gateway Interface) gunicorn, utilisé pour exécuter l'application Python en production.
- La création d'un fichier Procfile spécifiant la commande de démarrage, adaptée à l'hébergeur choisi (Railway).
- La gestion des variables d'environnement dans un fichier .env sécurisé, permettant d'isoler les informations sensibles (clés, identifiants, etc.).

≡ requirements.txt

```
1  Flask==2.3.2
2  Flask-SQLAlchemy==3.0.3
3  Flask-Login==0.6.2
4  Flask-Mail==0.9.1
5  python-dotenv==1.0.0
6  email-validator==1.3.1
7  Werkzeug==2.3.4
8  Jinja2==3.1.2
9  itsdangerous==2.1.2
10 click==8.1.3
```

⌘ Procfile

```
1  web: gunicorn app:app
```

```
.env
1 SECRET_KEY=[REDACTED]
2 DATABASE_URL=postgresql://postgres:[REDACTED]@localhost:5432/miaouff
3
4 DB_USER=postgres
5 DB_PASSWORD=[REDACTED]
6 DB_NAME=miaouff
7 DB_HOST=localhost
8 DB_PORT=5432
9
10 HF_API_KEY=[REDACTED]
11
12 ADMIN_EMAIL=julie@miaouff.fr
13 ADMIN_PASSWORD=[REDACTED]
14
15 MAIL_SERVER=smtp.gmail.com
16 MAIL_PORT=587
17 MAIL_USE_TLS=True
18 MAIL_USERNAME=julie@miaouff.fr
19 MAIL_PASSWORD=[REDACTED]
20 MAIL_DEFAULT_SENDER=julie@miaouff.fr
21
22 MONGODB_URI=mongodb+srv://julie:[REDACTED]@cluster0.t3w3esw.mongodb.net
23
```

Pour la partie déploiement, Railway a été retenu pour son intégration facile avec GitHub et la gestion simple des environnements. Le déploiement est réalisé directement à partir du dépôt Git, via un pipeline automatisé qui déploie la dernière version stable.

Enfin, une veille technologique a été mise en place pour suivre les évolutions des outils utilisés (Flask, SQLAlchemy, MongoDB, Railway), ainsi que pour se tenir informée des problématiques de sécurité liées au déploiement d'applications web. Cette veille est réalisée en suivant des sources spécialisées, des blogs de sécurité et des mises à jour des frameworks.

3. Contribuer à la mise en production dans une démarche DevOps

Dans le cadre de ce projet, une démarche d'intégration continue légère a été mise en place afin d'automatiser la vérification de la qualité du code et l'exécution des tests à chaque modification du dépôt.

Qualité du code

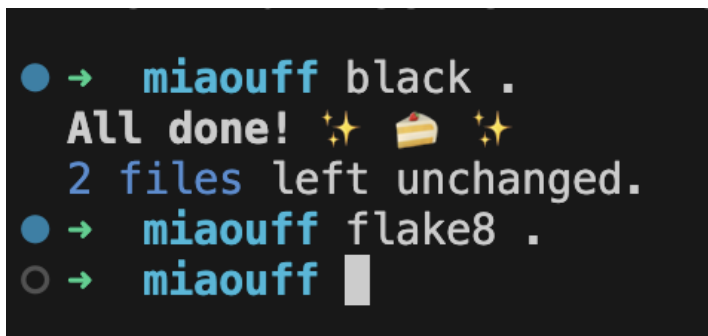
Deux outils sont utilisés systématiquement pour garantir la qualité du code Python :

- Flake8 : analyse statique vérifiant le respect des règles PEP8 et détectant les erreurs de style
- Black : formateur automatique assurant une cohérence de style sur l'ensemble du code

Ces outils sont exécutés localement avant chaque commit, et leur intégration dans le pipeline CI permet de bloquer automatiquement un merge si des violations sont détectées.

Vérification de la qualité du code

- Flake8 pour vérifier le respect des règles PEP8 et détecter les erreurs de style ou d'écriture.
- Black pour formater automatiquement le code et assurer une cohérence de style.



```
● → miaouff black .  
All done! ✨ 🍰 ✨  
2 files left unchanged.  
● → miaouff flake8 .  
○ → miaouff █
```

Intégration Continue (CI) avec GitHub Actions

Un workflow GitHub Actions a été configuré pour s'exécuter automatiquement à chaque push ou pull request sur la branche main. Le pipeline comprend les étapes suivantes :

- Installation des dépendances Python via `pip install -r requirements.txt`
- Vérification du style avec Flake8
- Formatage avec Black en mode `--check`
- Exécution des tests unitaires avec pytest

Extrait de CI.yaml :

DOSSIER PROFESSIONNEL (DP)

```
# CI/CD pipeline for Python backend with pytest
# Triggered on every push and pull request to main branch
name: CI Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    name: Run Tests
    runs-on: ubuntu-latest

    env:
      # Database URL injected from GitHub Secrets (never hardcode credentials)
      DATABASE_URL: ${ secrets.DATABASE_URL }
      # Added: MongoDB URI now injected into the CI environment
      MONGODB_URI: ${ secrets.MONGODB_URI }

    steps:
      # Step 1: Checkout the repository code
      - name: Checkout code
        uses: actions/checkout@v4

      # Step 2: Set up Python environment
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      # Step 3: Install project dependencies
      - name: Install dependencies
```

The screenshot displays the GitHub Actions interface for a repository named 'julie-carli / Miaouff'. The 'Actions' tab is selected, showing a list of workflow runs for the 'CI Pipeline' workflow. The interface includes a sidebar with navigation options like 'Code', 'Issues', 'Pull requests', 'Actions', 'Projects', 'Security and quality', 'Insights', and 'Settings'. The main area shows a table of workflow runs with columns for Workflow, Event, Status, Branch, and Actor. The table lists six runs, all of which are successful (green checkmarks) and occurred on the 'main' branch. The most recent run is 'fix css glossary' by 'julie-carli' on April 15, 2022, at 8:06 PM GMT+2, which completed in 32 seconds.

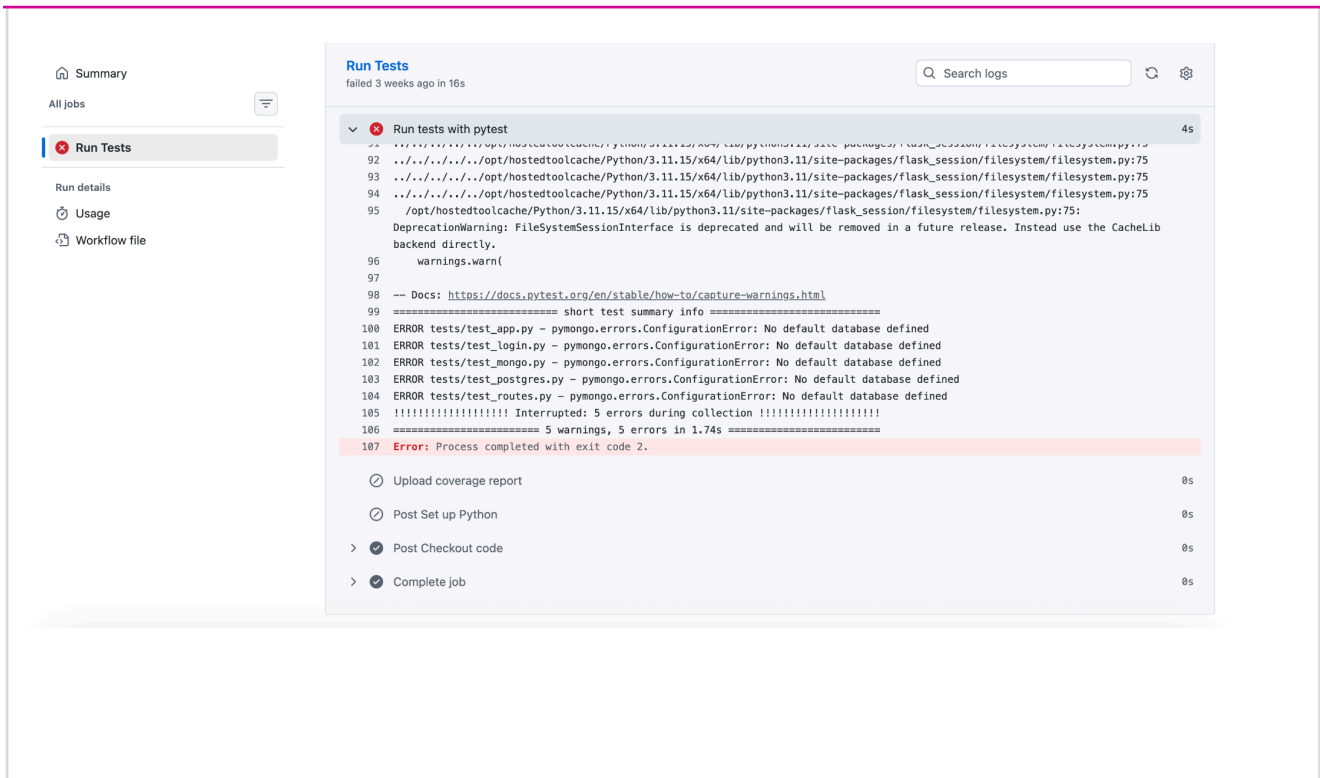
Workflow	Event	Status	Branch	Actor
fix css glossary	Push	Success	main	julie-carli
admin section design fixed and chatbot	Push	Success	main	julie-carli
fix account and add an eye for password	Push	Success	main	julie-carli
fix design main pages	Push	Success	main	julie-carli
Fix CI: inject MONGODB_URI secret into test environment	Push	Success	main	julie-carli
Add CI pipeline with GitHub Actions	Push	Success	main	julie-carli

Interprétation des rapports

Les rapports générés par le pipeline CI sont consultés après chaque exécution. En cas d'échec, le rapport indique précisément le fichier et la ligne concernés par l'erreur, ce qui permet d'identifier et de corriger rapidement le problème.

Exemple d'échec :

DOSSIER PROFESSIONNEL (DP)



2. Précisez les moyens utilisés :

Pour la réalisation du site web de Miaouff, j'ai utilisé l'éditeur VS Code et j'ai créé l'application en Python avec le framework Flask. J'ai aussi utilisé GitHub Actions et Railway avec Neon pour la base de données PostgreSQL.

3. Avec qui avez-vous travaillé ?

Projet réalisé en autonomie

4. Contexte

Nom de l'entreprise, organisme ou association ▶ Cliquez ici pour taper du texte.

Chantier, atelier, service ▶ Projet réalisé lors de la formation
Période d'exercice ▶ Du 29/09/2025 au 14/09/2026

5. Informations complémentaires (facultatif)

DOSSIER PROFESSIONNEL (DP)



Titres, diplômes, CQP, attestations de formation

(facultatif)

Intitulé	Autorité ou organisme	Date
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.

Déclaration sur l'honneur

Je soussignée Julie Carli ,
déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts et que je
suis l'auteure des réalisations jointes.

Fait à Lucé

le 14/09/2026

pour faire valoir ce que de droit.

Signature :



DOSSIER PROFESSIONNEL ^(DP)

Documents illustrant la pratique professionnelle

(facultatif)

Intitulé
Cliquez ici pour taper du texte.

DOSSIER PROFESSIONNEL ^(DP)

ANNEXES

(Si le RC le prévoit)

